ШПАРГАЛКА ДО ЗАХИСТУ ЗА 2 ГОДИНИ

Лабораторна робота №6: React.js (Home Page)

Предмет: Web-розробка **Варіант:** 8 (Helicopter / Вертольоти) **Стек:** React 18, JSX, Functional Components, CSS Modules

1. Головна суть: Чому React і що змінилося після 5-ї лаби?

Головна думка, яку треба сказати викладачу:

*«У 5-й лабораторній ми писали окремо HTML-розмітку у файлі `data/index.html`, а в JavaScript вручну шукали елементи через `document.getElementById` і склеювали рядки через `innerHTML` (**імперативний підхід**). У React ми перейшли на **декларативний підхід** та синтаксис **JSX**: тепер розмітка та логіка компонента описані разом в одному файлі JavaScript, а оновленням DOM займається сам React через механізм Virtual DOM».*

Критерій	Як було в Лабі 3–5 (Vanilla JS)	Як стало в Лабі 6 (React.js)
Файли верстки	Окремий великий `data/index.html` зі структурою тегів.	`public/index.html` містить лише `<div id="root"></div>`. Увесь HTML генерується в React.
Формування карток	Склеювання шаблонних рядків `items.map(h ⇒ ` ... `).join('')` та `innerHTML = ...`.	Функціональний компонент `<HelicopterCard helicopter={heli} />`, який повертає **JSX**.
Поділ коду	Глобальні скрипти (`dom.js`, `main.js`, `api.js`), які взаємодіють з глобальним DOM.	Компонентна архітектура: кожен візуальний блок (Header, Sidebar, Card) — це окремий незалежний компонент у власному файлі.
Стилізація	Окремі CSS файли, що підключалися через тег `<link>` в HTML.	Модульний імпорт CSS безпосередньо у відповідний JS-файл компонента (`import './Header.css'`).

2. Архітектура файлів нашого проєкту

За вимогою завдання інтерфейс розділено щонайменше на 5 окремих функціональних компонентів (кожен у своєму файлі):

1. Header (Шапка)

`src/components/Header/Header.jsx`

Бренд **MyHeli**, навігація (My Helicopters, Create), поле пошуку та кнопки Search/Clear.

2. Sidebar (Бокова панель)

`src/components/Sidebar/Sidebar.jsx`

Керування вертольотами: перемикач сортування (Sort toggle) та кнопка підрахунку пасажирів (Count).

3. CardsGrid (Сітка карток)

`src/components/CardsGrid/CardsGrid.jsx`

Контейнер секції контенту. Проходиться методом `.map()` по масиву даних вертольотів.

4. HelicopterCard (Картка вертольота)

`src/components/HelicopterCard/HelicopterCard.jsx`

Індивідуальна картка (235x180 прев'ю, назва, опис, характеристики, кнопки Edit/Remove).

5. Footer (Підвал сайту)

`src/components/Footer/Footer.jsx`

Інформація про систему MyHeli, дублюючі лінки та копірайт.

6. App (Головний контейнер)

`src/App.jsx`

Кореневий компонент, який імпортує дані вертольотів та збирає всі компоненти на одній сторінці.

3. Покроковий розбір коду (Що відбувається в кожному файлі)

3.1. Точка монтування: `public/index.html` та `src/index.js`

У `public/index.html` немає жодного нашого тегу, крім:

```
<div id="root"></div>
```

У `src/index.js` React підключається до цього контейнера:

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App';

// Створюємо корінь React 18 через новий метод createRoot
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Що сказати: `ReactDOM.createRoot` знаходить звичайний DOM-вузол з `id="root"` і бере над ним повний контроль. Метод `render(<App />)` перетворює дерево компонентів React на реальні DOM-вузли всередині цього блоку.

3.2. Головний компонент `src/App.jsx` та джерело даних

Дані про вертольоти зберігаються у `src/data/helicopters.js` (масив з 5 об'єктів з полями `id, name, passengers, maxSpeed, description`). Компонент `App` збирає всю сторінку:

```
import React from 'react';
import Header from './components/Header/Header';
import Sidebar from './components/Sidebar/Sidebar';
import CardsGrid from './components/CardsGrid/CardsGrid';
import Footer from './components/Footer/Footer';
import { INITIAL_HELICOPTERS } from './data/helicopters';
import './App.css';

function App() {
  return (
    <div className="app">
      <Header />
      <main className="main-container">
        <Sidebar totalPassengers="—" isSorted={false} />
        {/* Передаємо масив вертольотів дочірньому компоненту через props */}
        <CardsGrid helicopters={INITIAL_HELICOPTERS} />
      </main>
      <Footer />
    </div>
  );
}

export default App;
```

3.3. Рендеринг списку: `src/components/CardsGrid/CardsGrid.jsx`

```
import React from 'react';
import HelicopterCard from '../HelicopterCard/HelicopterCard';
import './CardsGrid.css';

const CardsGrid = ({ helicopters = [] }) => {
  return (
    <section className="content">
      <div className="cards-grid" id="cards-grid">
        {helicopters.length === 0 ? (
          <div className="cards-empty">Вертольотів не знайдено.</div>
        ) : (
          /* Метод .map() перетворює кожен JS-об'єкт вертольота на JSX-компонент */
          helicopters.map((heli) => (
            <HelicopterCard key={heli.id} helicopter={heli} />
          ))
        )}
      </div>
    </section>
  );
};

export default CardsGrid;
```

3.4. Відображення одного вертольота: `src/components/HelicopterCard/HelicopterCard.jsx`

```
import React from 'react';
import './HelicopterCard.css';

const HelicopterCard = ({ helicopter }) => {
  // Деструктуризація об'єкта пропса для зручного звернення
  const { id, name, description, maxSpeed, passengers } = helicopter;

  return (
    <article className="card" data-id={id}>
      <div className="card__image">
        <span className="card__image-text">235x180</span>
      </div>
      <div className="card__body">
        <h3 className="card__title">{name}</h3>
        <p className="card__desc">{description}</p>
        <p className="card__specs">
          <strong>Макс. швидкість:</strong> {maxSpeed} км/год<br />
          <strong>Пасажири:</strong> {passengers} місць
        </p>
        <div className="card__updated">Last updated 3 mins ago</div>
        <div className="card__actions">
          <button type="button" className="btn-card-edit">Edit</button>
          <button type="button" className="btn-card-remove">Remove</button>
        </div>
      </div>
    </article>
  );
};

export default HelicopterCard;
```

4. ТОП-10 питань викладача на захисті та готові відповіді

ПИТАННЯ 1 **Що таке JSX і чому браузер не може виконати його напряму?**

Відповідь: JSX (JavaScript XML) — це розширення синтаксису JavaScript, яке дозволяє писати HTML-подібну розмітку безпосередньо у коді JS. Браузер не розуміє JSX напряму, тому транспілятор **Babel** (у складі `react-scripts`) перед виконанням перетворює JSX на стандартні виклики функцій `React.createElement(...)` або новий JSX runtime `_jsx(...)`.

ПИТАННЯ 2 **Чому замість class ми пишемо className, а замість for — htmlFor?**

Відповідь: Оскільки JSX — це розширення мови JavaScript, слова `class` та `for` є зарезервованими ключовими словами у JS (для оголошення класів та циклів). Щоб уникнути синтаксичного конфлікту, у React для HTML-атрибутів використовуються відповідні DOM-властивості: `className` та `htmlFor`.

ПИТАННЯ 3 **Що таке Функціональні Компоненти (Functional Components)? Чим вони кращі за класові?**

Відповідь: Функціональний компонент — це звичайна JS-функція, яка приймає вхідні дані (`props`) і повертає JSX. Вони є сучасним стандартом React (починаючи з версії 16.8), оскільки мають значно простіший та лаконічніший синтаксис, не потребують ключового слова `this`, займають менше пам'яті та легко використовують React Hooks (`useState`, `useEffect`).

ПИТАННЯ 4 **Що таке Props (пропси) і яке правило їх зміни в React?**

Відповідь: Props (properties) — це вхідні параметри, які передаються від батьківського компонента до дочірнього (наприклад, `<CardsGrid helicopters={INITIAL_HELICOPTERS} />`).
Важливе правило: Props є **read-only (незмінними)**! Дочірній компонент ніколи не повинен мутувати (змінювати) свої пропси напряму, оскільки React підтримує строгий односторонній потік даних (One-Way Data Flow).

ПИТАННЯ 5 **Навіщо обов'язково вказувати key={heli.id} у циклі map()?**

Відповідь: Ключ `key` — це унікальний ідентифікатор елемента списку, який потрібен алгоритму узгодження React (**Reconciliation / Virtual DOM diffing**). Завдяки `key` React точно знає, яка саме картка була додана, змінена чи видалена, і оновлює в реальному браузері лише цей конкретний вузол, а не перемальовує весь список заново.

ПИТАННЯ 6 **Що таке Virtual DOM (Віртуальний DOM) і як він працює?**

Відповідь: Virtual DOM — це легка копія реального дерева DOM у пам'яті (у вигляді простих JS-об'єктів). Коли змінюються дані:

1. React будує нове дерево Virtual DOM.
2. Порівнює його зі старим деревом (процес **Diffing**).
3. Знаходить мінімальну необхідну різницю та точково вносить зміни в реальний браузерний DOM (процес **Reconciliation**). Це на порядки швидше за пряме перезаписування `innerHTML`.

ПИТАННЯ 7 **Чому в index.html нічого немає, а сайт відображається?**

Відповідь: Тому що це архітектура **Single Page Application (SPA)**. Браузер завантажує порожній скелет сторінки з єдиним тегом `<div id="root">` і скрипт збірки React. Скрипт виконується на клієнті і за допомогою `ReactDOM.createRoot().render(<App />)` повністю формує весь HTML і вставляє його в `root`.

ПИТАННЯ 8 **Як передаються стилі компонентів у нашому додатку?**

Відповідь: Кожен компонент має свій окремий CSS-файл (наприклад, `Header.css`, `Sidebar.css`, `CardsGrid.css`), який імпортується безпосередньо у відповідний файл компонента через команду `import './Header.css'`. Під час збірки Webpack збирає ці стилі в єдиний оптимізований бандл.

ПИТАННЯ 9 **Які дані представляє наша сутність (Варіант 8)?**

Відповідь: Сутність — **Helicopter (Вертоліт)**. Поля об'єкта відповідають попереднім роботам: `id` (число), `name` (назва вертольота, рядок), `passengers` (кількість пасажирських місць, число), `maxSpeed` (максимальна швидкість у км/год, число), `description` (опис призначення).

ПИТАННЯ 10 **Чому кнопки Edit, Remove та Search зараз не виконують дії?**

Відповідь: Згідно з технічним завданням 6-ї лабораторної роботи, вимагається реалізувати **виключно представлення сторінки (View-only part)**: перенести розмітку в компоненти React і налаштувати стилі. Вся інтерактивність (useState, фільтрація, роутинг, редагування через форми та робота з API) запланована на наступні лабораторні роботи (7–8).

5. Експрес-пам'ятка за 2 хвилини до захисту

Якщо викладач запитає «Розкажи коротко свій код»:

Говори за цією схемою (30 секунд):

1. «У цій лабораторній роботі ми перенесли наш сайт каталогу вертольотів з ванільного JS на **React 18**».
2. «Ми повністю зберегли верстку та дизайн із 5-ї лаби (шапка `MyHeli`, бокове меню сортування та сітка карток), але тепер весь UI розбитий на **5 окремих функціональних компонентів**: `Header`, `Sidebar`, `CardsGrid`, `HelicopterCard` та `Footer`».
3. «У `index.html` розмітка чиста (тільки `div id="root"`). Точка входу `index.js` монтує кореневий компонент `App` за допомогою `createRoot`».
4. «Дані вертольотів зберігаються у масиві `INITIAL_HELICOPTERS` і передаються через `props` у `CardsGrid`, де за допомогою методу `.map()` рендеряться окремі картки `HelicopterCard` з унікальним `key={heli.id}`».
5. «Усі компоненти функціональні, без класового `this`. Згідно з вимогами лаби 6, реалізовано представлення (view-only), а стан та взаємодію буде додано в наступних роботах».
